Politecnico di Milano - sede di Cremona

Ingegneria del Software – a.a. 2005/06

Appello del 8 settembre 2006

Cognome
Nome
Matricola

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 1h30min.
6. Punteggio totale a disposizione: 25/30. La sufficienza si raggiunge con 16 punti, a cui sommare il risultato delle prove di laboratorio.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Totale .....

Esercizio 1 (punti 10)

L'ADT immutabile Carta fornisce un'astrazione per una carta da briscola: asso ("1"), due ("2"), tre ("3"), quattro ("4"), cinque ("5"), sei ("6"), sette ("7"), fante ("F"), cavallo ("C") e re ("R") di spade ("S"), coppe ("C"), bastoni ("B") e denari ("D") per un totale di quaranta possibili carte distinte. L'ADT *MazzoCarte* fornisce un'astrazione per un mazzo di carte da briscola, ossia di quaranta carte.

Un giocatore può pescare una carta dal mazzo usando il metodo *pesca*(). Questo metodo restituisce una carta (a caso) e la elimina dal mazzo. Quando un giocatore pesca l'ultima carta del mazzo, il metodo solleva anche ---dopo aver restituito la carta e dopo averla eliminata dal mazzo--- l'eccezione ExceptionMazzoVuoto. L'ADT fornisce anche i seguenti i metodi puri *nelMazzo* e *carteDelMazzo*:

```
//@ requires true ;
//@ ensures (*restituisce true se c è parte del mazzo*) ;
public /*@pure@*/ boolean nelMazzo(Carta c)
```

```
//@ requires true ;
//@ ensures (*restituisce il numero di carte rimaste nel mazzo*) ;
public /*@pure@*/ int carteNelMazzo()
```

a- Si scriva la specifica del metodo pesca tenendo presente che:

- Si assume che il mazzo debba contenere almeno una carta
- Il numero di carte nel mazzo viene decrementato di uno
- La carta pescata non è più presente nel mazzo
- La carta pescata è l'unica carta eliminata dal mazzo

```
//@ requires
```

```
//@ ensures
```

```
public Carta pesca()
```

```
public Carta pesca()
Soluzione:
requires carteNelMazzo() >=1
ensures     \old(nelMazzo(\result)) &&
            !nelMazzo(\result) &&
            carteNelMazzo() == \old(carteNelMazzo()) -1 &&
            (\forall Carta c; c !=\result;nelMazzo(c) <==> \old(nelMazzo(c)));
```

b- Si trasformi la specifica del metodo pesca eliminando l'assuzione che il mazzo non sia vuoto e includendo il lancio di un'eccezione ExceptionMazzoVuoto. Si mostri solo la parte di specifica modificata rispetto a quella definita in (a).

```
//@ requires
```

//@ ensures

public Carta pesca() throws MazzoVuotoException

Soluzione:
requires true;
Aggiungere alla ensures: \old(carteNelMazzo() >=1)
Aggiungere la clausola:
signals(MazzoVuotoException e) \old(carteNelMazzo() ==0) ;

c- Sapendo che l'ADT *Carta* fornisce i seguenti metodi puri per conoscere il valore e il seme di una carta:

//@ requires true ;
//@ ensures (*restituisce il valore di this*) ;
public /*@pure@*/ char valore()

//@ requires true ;
//@ ensures (*restituisce il seme di this*)
public /*@pure@*/ char seme()

Si scriva l'invariante pubblico per la classe MazzoCarte per imporre che un mazzo non può avere carte ripetute.
//@public invariant

Soluzione: non ci possono essere due reference c1 e c2, c1 != c2, a carte con stesso valore e seme:
public invariant
(\forall Carta c1; nelMazzo(c1);
(\forall Carta c2; nelMazzo(c2); c1.valore()!= c2.valore() || c1.seme()!= c2.seme() || c1==c2)));

Esercizio 2 (punti 7)

Supponendo che sia definita la classe :

public class InsiemeInteri
private int[] insieme;

si scriva in Java un opportuno iteratore che restituisca in sequenza i numeri primi contenuti nell'array, definendo sia il metodo *primi()* di InsiemeNumeri che restituisce l'iteratore, sia la classe *PrimiGen* che definisce l'iteratore (ossia il generatore). L'iteratore deve scandire gli elementi dell'array e restituire solo i numeri primi. Ad esempio, se l'array contenesse i numeri 5, 7, 23, 45, 13, 48, 17, l'iteratore dovrebbe restituire i numeri 5, 7, 23, 13, e 17.

public Iterator <Integer> primi() {
.....
.....
.....
}

Classe del generatore :
private static class PrimiGen .....

}

Soluzione :

```
public Iterator <Integer> primi() {
  return new PrimiGen(this) ;
}

private static class PrimiGen implements Iterator<Integer> {
  private InsiemeInteri i ;
  private int n ;
  PrimiGen(InsiemeInteri ins){
    i=ins ;
    n=0 ;
    while(n<i.insieme.length && !isPrime(i.insieme[n])) n++ ;
                          /* isPrime metodo statico implementato in modo ovvio */
  }

  public boolean hasNext() { return n<i.insieme.length ;}

  public Integer next() throws NoSuchElementException {
    Integer res ;
    if (this.hasNext()) res = i,insieme[n] ;
    else throw new NoSuchElementException("PrimiGen") ;
    while(n<i.insieme.length && !isPrime(i.insieme[n])) n++ ;
    return res ;
  }
```

Esercizio 3 (punti 8)

Si consideri una classe Java MiaClasse:

```
public class MiaClasse {
   protected static int num = 0;
   protected int val;
  public MiaClasse(int a){
     val = a;
     num += a;
   }
}
```

a- Supponendo di creare due oggetti o1 e o2 in sequenza:

```
o1 = new MiaClasse(5);
o2 = new MiaClasse(3);
```

quali valori assumono i campi val e num dei due oggetti?

Sol.
o1 = new MiaClasse(5); val = 5 e num = 5
o2 = new MiaClasse(3); val = 3 e num = 8

b- Aggiungendo due sottoclassi MiaSottoclasse1 e MiaSottoclasse2:

```
public class MiaSottoclasse1 extends MiaClasse {…}
public class MiaSottoclasse2 extends MiaClasse {…}
```

```
MiaClasse a;
MiaSottoclasse1 b;
MiaSottoclasse2 c;
```

Quali delle seguenti righe di codice darebbe un errore in compilazione? (mettere una X per ogni riga che si ritiene scorretta aggiungendo una breve spiegazione)

```
a = new MiaClasse();
b = new MiaClasse();
c = new MiaSottoclasse1();
b = a;
a = b;
b = new MiaSottoclasse2();
```

Sol.
a = new MiaClasse();
b = new MiaClasse(); X assegnamento di un oggetto a una variabile di una sottoclasse della sua classe
c = new MiaSottoclasse1(); X assegnamento di un oggetto a una variabile di una classe non antenato della sua classe
b = a; X assegnamento di una variabile a una variabile di una sottoclasse della sua classe
a = b;
b = new MiaSottoclasse2(); X assegnamento di un oggetto a una variabile di una classe non antenato della sua classe